Deep Reinforcement Learning

Bonus Unit 1: Train and Play with Huggy

Hugging Face Deep RL Course

Complete MDX-to-LaTeX study edition

Source: Hugging Face Deep RL Course (original English text).

Contents

1 Introduction **2**

2 How Huggy works **2**

2.1 The State Space, what Huggy perceives. . . . 2
2.2 The Action Space, what moves Huggy can perform . . . 3
2.3 The Reward Function . . . 3
2.4 Train Huggy . . . 4

3 Let's train and play with Huggy **5**

3.1 Let's train Huggy . . . 5
3.2 Objectives of this notebook . . . 6
3.3 Prerequisites . . . 7
3.4 Set the GPU . . . 7
3.5 Clone the repository . . . 7
3.6 Setup the Virtual Environment . . . 8
3.7 Installing the dependencies . . . 9
3.8 Download and move the environment zip file . . . 9
3.9 Let's recap how this environment works . . . 9
3.10 Check the Huggy config file . . . 11
3.11 Train our agent . . . 12
3.12 Push the agent to the Hub . . . 12
3.13 Play with your Huggy . . . 14
3.14 Keep Learning, Stay awesome . . . 15

4 Play with Huggy **16**

5 Conclusion **16**

1 Introduction

In this bonus unit, we'll reinforce what we learned in the first unit by teaching Huggy the Dog to fetch the stick and then play with him directly in your browser



Figure 1: Hugging Face Deep RL Course: Bonus Unit 1, Huggy [original figure]

So let's get started

2 How Huggy works

Huggy is a Deep Reinforcement Learning environment made by Hugging Face and based on Puppo the Corgi, a project by the Unity MLAgents team. This environment was created using the Unity game engine and MLAgents. ML-Agents is a toolkit for the game engine from Unity that allows us to **create environments using Unity or use pre-made environments to train our agents**.



Figure 2: Huggy: the Unity ML-Agents dog environment [original figure]

In this environment we aim to train Huggy to **fetch the stick we throw. This means he needs to move correctly toward the stick**.

2.1 The State Space, what Huggy perceives.

Huggy doesn't "see" his environment. Instead, we provide him information about the environment:

- The target (stick) position
- The relative position between himself and the target
- The orientation of his legs.

Given all this information, Huggy can **use his policy to determine which action to take next to fulfill his goal**.

2.2 The Action Space, what moves Huggy can perform

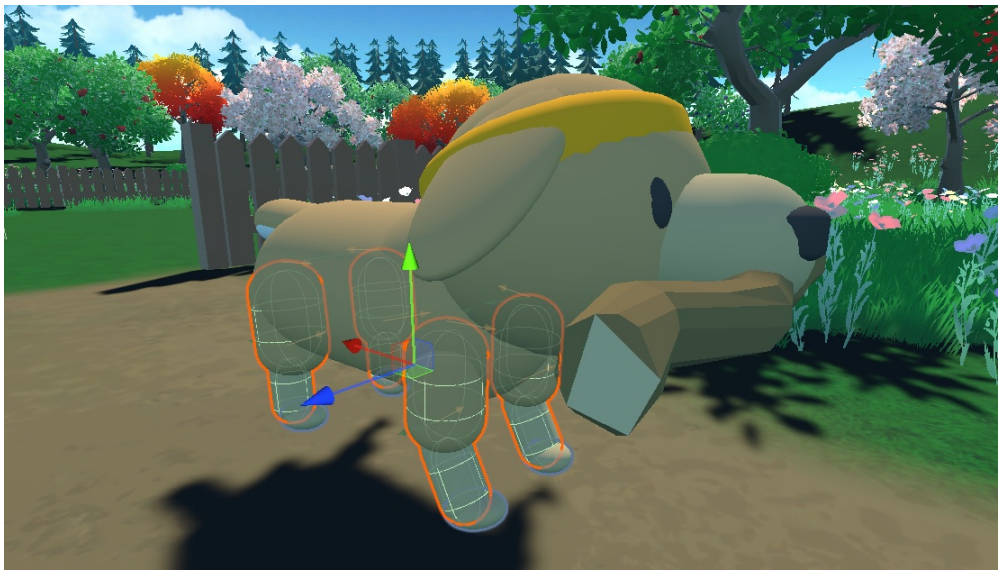


Figure 3: Joint-motor actions controlling Huggy's legs [original figure]

Joint motors drive Huggy's legs. This means that to get the target, Huggy needs to **learn to rotate the joint motors of each of his legs correctly so he can move**.

2.3 The Reward Function

The reward function is designed so that **Huggy will fulfill his goal**: fetch the stick.

Remember that one of the foundations of Reinforcement Learning is the *reward hypothesis*: a goal can be described as the **maximization of the expected cumulative reward**.

Here, our goal is that Huggy **goes towards the stick but without spinning too much**. Hence, our reward function must translate this goal.

Our reward function:



Figure 4: The reward function used to train Huggy [original figure]

- *Orientation bonus*: we **reward him for getting close to the target**.
- *Time penalty*: a fixed-time penalty given at every action to **force him to get to the stick as fast as possible**.
- *Rotation penalty*: we penalize Huggy if **he spins too much and turns too quickly**.
- *Getting to the target reward*: we reward Huggy for **reaching the target**.

If you want to see what this reward function looks like mathematically, check Puppo the Corgi presentation.

2.4 Train Huggy

Huggy aims **to learn to run correctly and as fast as possible toward the goal**. To do that, at every step and given the environment observation, he needs to decide how to rotate each joint motor of his legs to move correctly (not spinning too much) and towards the goal.

The training loop looks like this:

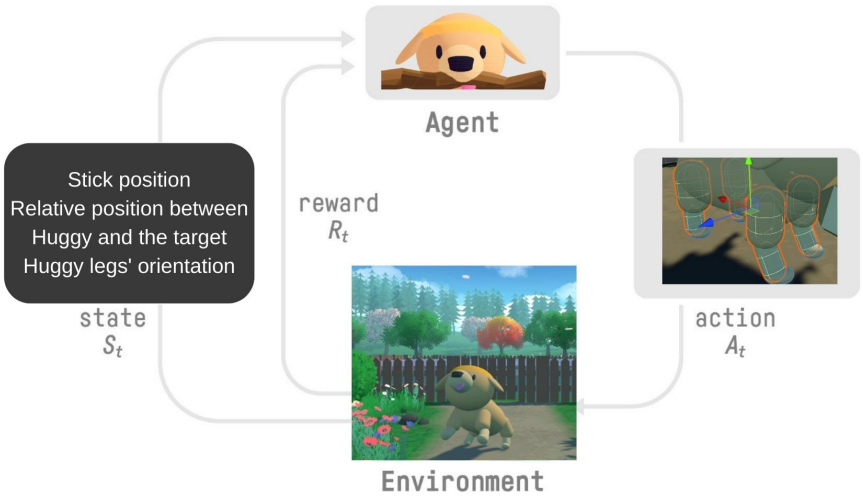


Figure 5: The interaction and training loop for Huggy [original figure]

The training environment looks like this:

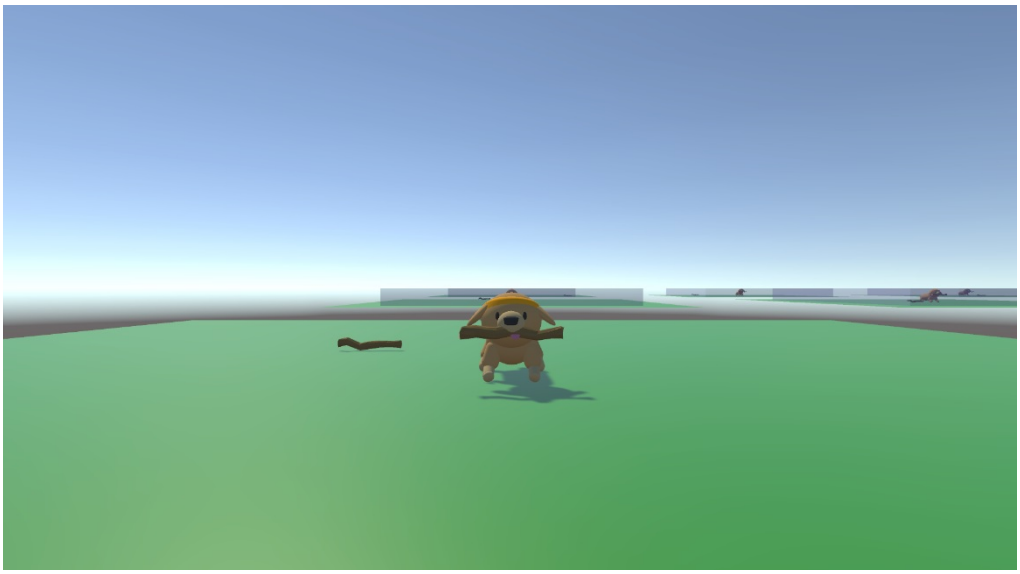


Figure 6: Parallel training arenas with randomized stick positions [original figure]

It's a place where a **stick is spawned randomly**. When Huggy reaches it, the stick get spawned somewhere else. We built **multiple copies of the environment for the training**. This helps speed up the training by providing more diverse experiences.

Now that you have the big picture of the environment, you're ready to train Huggy to fetch the stick.

To do that, we're going to use MLAgents. Don't worry if you have never used it before. In this unit we'll use Google Colab to train Huggy, and then you'll be able to load your trained Huggy and play with him directly in the browser.

In a future unit, we will study MLAgents more in-depth and see how it works. But for now, we keep things simple by just using the provided implementation.

3 Let's train and play with Huggy

Open the bonus-unit training notebook in Google Colab

We strongly **recommend students use Google Colab for the hands-on exercises** instead of running them on their personal computers.

By using Google Colab, **you can focus on learning and experimenting without worrying about the technical aspects** of setting up your environments.

3.1 Let's train Huggy

To start to train Huggy, click on Open In Colab button :

Open in Google Colab



Figure 7: Hugging Face Deep RL Course: Bonus Unit 1, Huggy [original figure]

In this notebook, we'll reinforce what we learned in the first Unit by **teaching Huggy the Dog to fetch the stick and then play with it directly in your browser**



Figure 8: Huggy: the Unity ML-Agents dog environment [original figure]

The environment

- Huggy the Dog, an environment created by Thomas Simonini based on Puppo The Corgi

The library used

- MLAgents

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the Github Repo.

3.2 Objectives of this notebook

At the end of the notebook, you will:

- Understand **the state space, action space, and reward function used to train Huggy**.
- **Train your own Huggy** to fetch the stick.
- Be able to play **with your trained Huggy directly in your browser**.

3.3 Prerequisites

Before diving into the notebook, you need to:

[ ] **Develop an understanding of the foundations of Reinforcement learning** (MC, TD, Rewards hypothesis...) by doing Unit 1

[ ] **Read the introduction to Huggy** by doing Bonus Unit 1

3.4 Set the GPU

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`

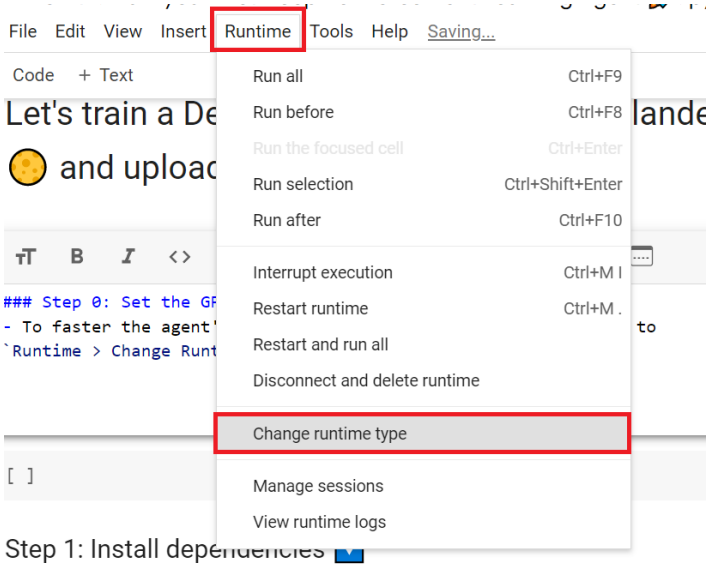


Figure 9: Google Colab: changing the runtime type [original figure]

- `Hardware Accelerator > GPU`

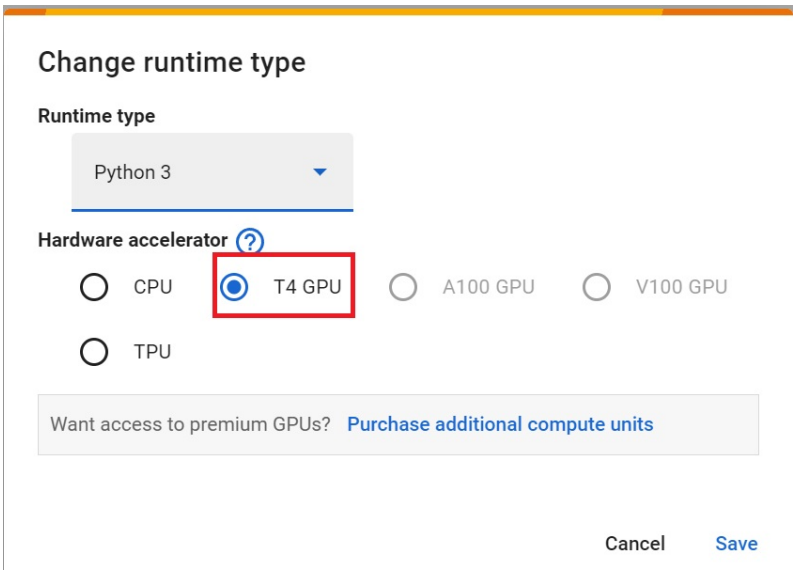


Figure 10: Google Colab: selecting a GPU accelerator [original figure]

3.5 Clone the repository

- We need to clone the repository, that contains **ML-Agents.**

```
# Clone the repository (can take 3min)
git clone --depth 1 https://github.com/Unity-Technologies/ml-agents
```

3.6 Setup the Virtual Environment

- In order for the **ML-Agents** to run successfully in Colab, Colab's Python version must meet the library's Python requirements.
- We can check for the supported Python version under the `python_requires` parameter in the `setup.py` files. These files are required to set up the **ML-Agents** library for use and can be found in the following locations:
  - `/content/ml-agents/ml-agents/setup.py`
  - `/content/ml-agents/ml-agents-envs/setup.py`
- Colab's Current Python version(can be checked using `!python --version`) doesn't match the library's `python_requires` parameter, as a result installation may silently fail and lead to errors like these, when executing the same commands later:
  - `/bin/bash: line 1: mlagents-learn: command not found`
  - `/bin/bash: line 1: mlagents-push-to-hf: command not found`
- To resolve this, we'll create a virtual environment with a Python version compatible with the **ML-Agents** library.

`Note:` *For future compatibility, always check the* ***`python_requires`*** *parameter in the installation files and set your virtual environment to the maximum supported Python version in the given below script if the Colab's Python version is not compatible*

```
# Colab's Current Python Version (Incompatible with ML-Agents)
!python --version
```

```
# Install virtualenv and create a virtual environment
!pip install virtualenv
!virtualenv myenv

# Download and install Miniconda
!wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
!chmod +x Miniconda3-latest-Linux-x86_64.sh
!./Miniconda3-latest-Linux-x86_64.sh -b -f -p /usr/local

# Activate Miniconda and install Python ver 3.10.12
!source /usr/local/bin/activate
!conda install -q -y --prefix /usr/local python=3.10.12 ujson  # Specify the version here

# Set environment variables for Python and conda paths
!export PYTHONPATH=/usr/local/lib/python3.10/site-packages/
!export CONDA_PREFIX=/usr/local/envs/myenv
```

```
# Python Version in New Virtual Environment (Compatible with ML-Agents)
!python --version
```

3.7 Installing the dependencies

```
# Go inside the repository and install the package (can take 3min)
%cd ml-agents
pip3 install -e ./ml-agents-envs
pip3 install -e ./ml-agents
```

3.8 Download and move the environment zip file

- Our environment executable is in a zip file.
- We need to download it and place it to `./trained-envs-executables/linux/`

```
mkdir ./trained-envs-executables
mkdir ./trained-envs-executables/linux
```

We downloaded the file Huggy.zip from https://github.com/huggingface/Huggy using `wget`

```
wget "https://github.com/huggingface/Huggy/raw/main/Huggy.zip" -O ./trained-envs-executables/linux/
    Huggy.zip
```

```
%%capture
unzip -d ./trained-envs-executables/linux/ ./trained-envs-executables/linux/Huggy.zip
```

Make sure your file is accessible

```
chmod -R 755 ./trained-envs-executables/linux/Huggy
```

3.9 Let's recap how this environment works

The State Space: what Huggy perceives.

Huggy doesn't "see" his environment. Instead, we provide him information about the environment:

- The target (stick) position
- The relative position between himself and the target
- The orientation of his legs.

Given all this information, Huggy **can decide which action to take next to fulfill his goal**.



Figure 11: Huggy: the Unity ML-Agents dog environment [original figure]

The Action Space: what moves Huggy can do

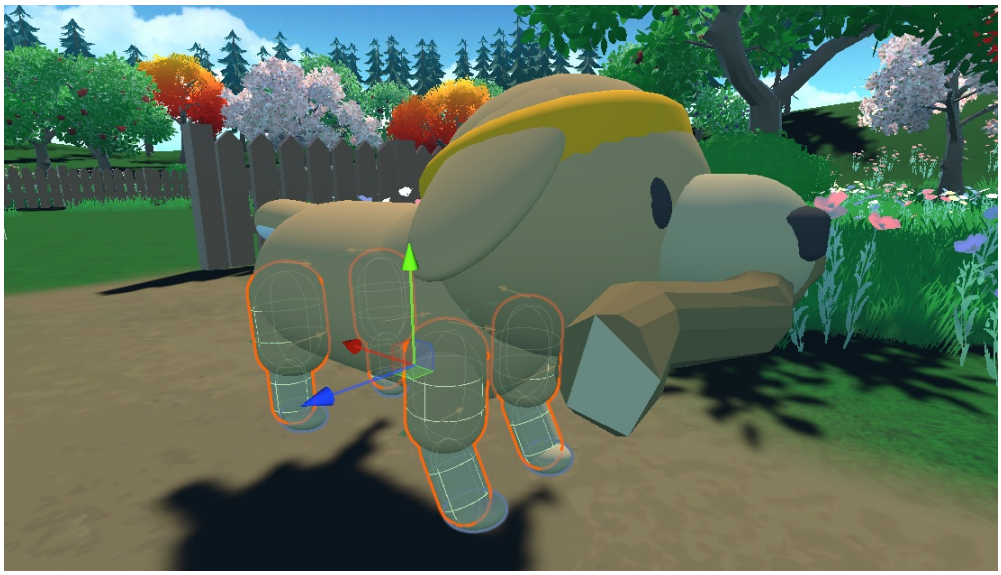


Figure 12: Joint-motor actions controlling Huggy's legs [original figure]

Joint motors drive huggy legs. This means that to get the target, Huggy needs to **learn to rotate the joint motors of each of his legs correctly so he can move**.

The Reward Function

The reward function is designed so that **Huggy will fulfill his goal** : fetch the stick.

Remember that one of the foundations of Reinforcement Learning is the *reward hypothesis*: a goal can be described as the **maximization of the expected cumulative reward**.

Here, our goal is that Huggy **goes towards the stick but without spinning too much**. Hence, our reward function must translate this goal.

Our reward function:



Figure 13: The reward function used to train Huggy [original figure]

- *Orientation bonus*: we **reward him for getting close to the target**.
- *Time penalty*: a fixed-time penalty given at every action to **force him to get to the stick as fast as possible**.
- *Rotation penalty*: we penalize Huggy if **he spins too much and turns too quickly**.
- *Getting to the target reward*: we reward Huggy for **reaching the target**.

3.10 Check the Huggy config file

- In ML-Agents, you define the **training hyperparameters in config.yaml files.**
- For the scope of this notebook, we're not going to modify the hyperparameters, but if you want to try as an experiment, Unity provides very good documentation explaining each of them here.
- We need to create a config file for Huggy.
- Go to `/content/ml-agents/config/ppo`
- Create a new file called `Huggy.yaml`
- Copy and paste the content below

```
behaviors:
  Huggy:
    trainer_type: ppo
    hyperparameters:
      batch_size: 2048
      buffer_size: 20480
      learning_rate: 0.0003
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
      learning_rate_schedule: linear
    network_settings:
      normalize: true
      hidden_units: 512
      num_layers: 3
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.995
        strength: 1.0
    checkpoint_interval: 200000
```

```
    keep_checkpoints: 15
    max_steps: 2e6
    time_horizon: 1000
    summary_freq: 50000
```

- Don't forget to save the file!
- **In the case you want to modify the hyperparameters**, in Google Colab notebook, you can click here to open the config.yaml: `/content/ml-agents/config/ppo/Huggy.yaml`

We're now ready to train our agent .

3.11 Train our agent

To train our agent, we just need to **launch mlagents-learn and select the executable containing the environment.**

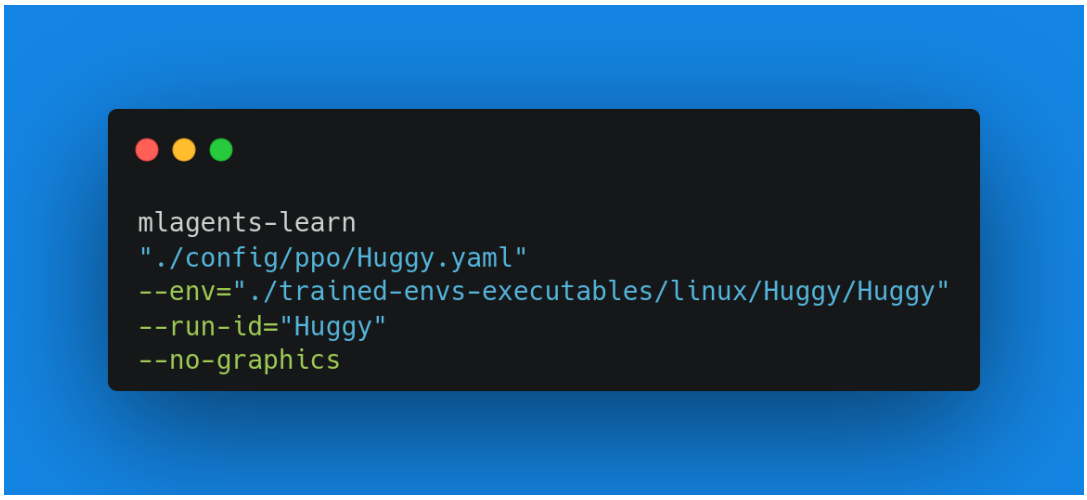


Figure 14: Structure of the mlagents-learn command and its options [original figure]

With ML Agents, we run a training script. We define four parameters:

1. `mlagents-learn <config>`: the path where the hyperparameter config file is.
2. `--env`: where the environment executable is.
3. `--run-id`: the name you want to give to your training run id.
4. `--no-graphics`: to not launch the visualization during the training.

Train the model and use the `--resume` flag to continue training in case of interruption.

> It will fail first time when you use `--resume`, try running the block again to bypass the error.

The training will take 30 to 45min depending on your machine (don't forget to **set up a GPU**), go take a coffee you deserve it .

```
mlagents-learn ./config/ppo/Huggy.yaml --env=./trained-envs-executables/linux/Huggy/Huggy --run-id
    ="Huggy" --no-graphics
```

3.12 Push the agent to the Hub

- Now that we trained our agent, we're **ready to push it to the Hub to be able to play with Huggy on your browser.**

To be able to share your model with the community there are three more steps to follow:

1. (If it's not already done) create an account to HF -> https://huggingface.co/join
2. Sign in and then get your token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**

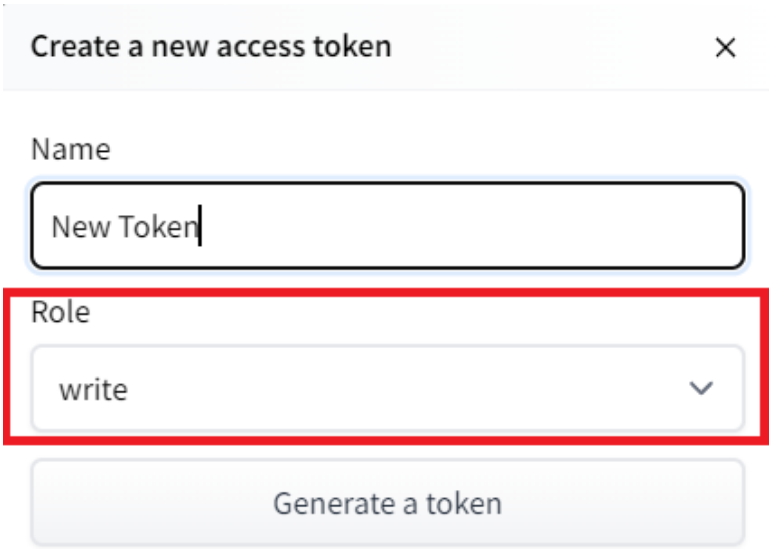


Figure 15: Creating a Hugging Face access token with write permission [original figure]

- Copy the token
- Run the cell below and paste the token

```
from huggingface_hub import notebook_login

notebook_login()
```

If you don't want to use Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

Then, we simply need to run `mlagents-push-to-hf`.



Figure 16: Structure of the mlagents-push-to-hf command [original figure]

And we define 4 parameters:

1. `--run-id`: the name of the training run id.
2. `--local-dir`: where the agent was saved, it's `results/<run_id name>`, so in my case results/First Training.
3. `--repo-id`: the name of the Hugging Face repo you want to create or update. It's always `<your huggingface username>/<the repo name>` If the repo does not exist **it will be created automatically**
4. `--commit-message`: since HF repos are git repositories you need to give a commit message.

```
mlagents-push-to-hf --run-id="HuggyTraining" --local-dir="./results/Huggy" --repo-id="
    ThomasSimonini/ppo-Huggy" --commit-message="Huggy"
```

If everything worked you should see this at the end of the process (but with a different url) :

```
Your model is pushed to the hub. You can view your model here: https://huggingface.co/
    ThomasSimonini/ppo-Huggy
```

It's the link to your model repository. The repository contains a model card that explains how to use the model, your Tensorboard logs and your config file. **What's awesome is that it's a git repository, which means you can have different commits, update your repository with a new push, open Pull Requests, etc.**

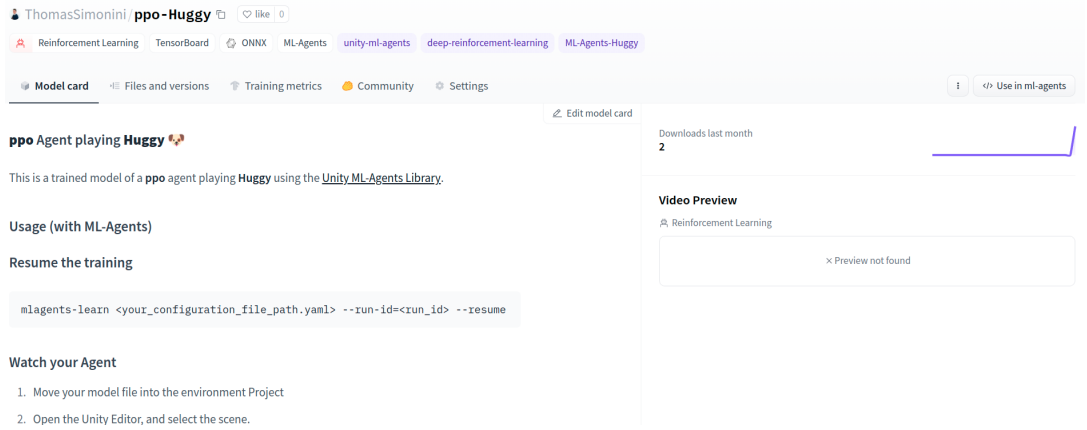


Figure 17: Example model repository and Model Card after uploading Huggy [original figure]

But now comes the best part: **being able to play with Huggy online .**

3.13 Play with your Huggy

This step is the simplest:

- Open the Huggy game in your browser: https://huggingface.co/spaces/ThomasSimonini/Huggy
- Click on Play with my Huggy model

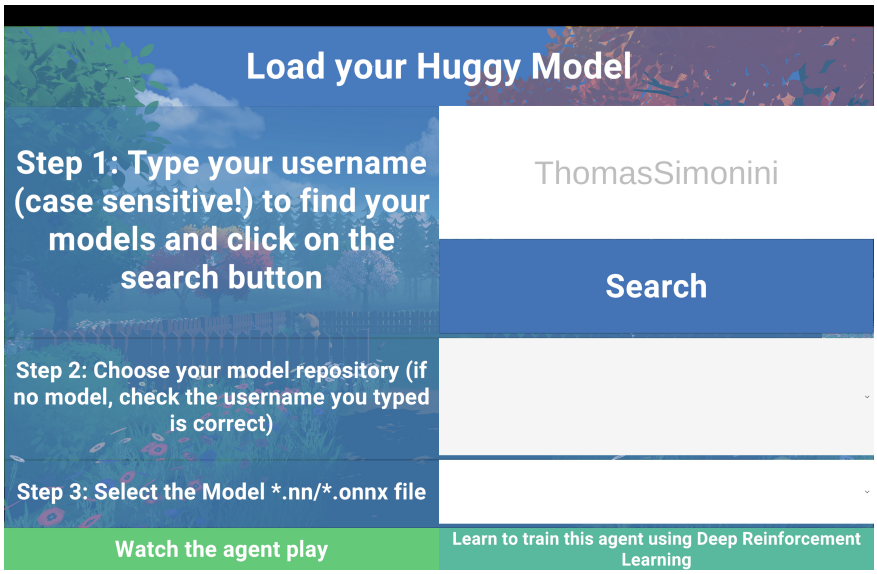


Figure 18: The Huggy browser demo: selecting a model from your account [original figure]

1. In step 1, type your username (your username is case sensitive: for instance, my username is ThomasSimonini not thomassimonini or ThOmasImoNInI) and click on the search button.
2. In step 2, select your model repository.
3. In step 3, **choose which model you want to replay**:

- I have multiple ones, since we saved a model every 500000 timesteps.
- But since I want the most recent one, I choose `Huggy.onnx`

It's good **to try with different models steps to see the improvement of the agent.**

Congrats on finishing this bonus unit!

You can now sit and enjoy playing with your Huggy . And don't **forget to spread the love by sharing Huggy with your friends** . And if you share about it on social media, **please tag us @huggingface and me @simoninithomas**



Figure 19: Huggy the Dog in the game environment [original figure]

3.14 Keep Learning, Stay awesome

4 Play with Huggy

Now that you've trained Huggy and pushed it to the Hub. **You will be able to play with him**

For this step it's simple:

- Open the Huggy game in your browser: https://huggingface.co/spaces/ThomasSimonini/Huggy
- Click on Play with my Huggy model

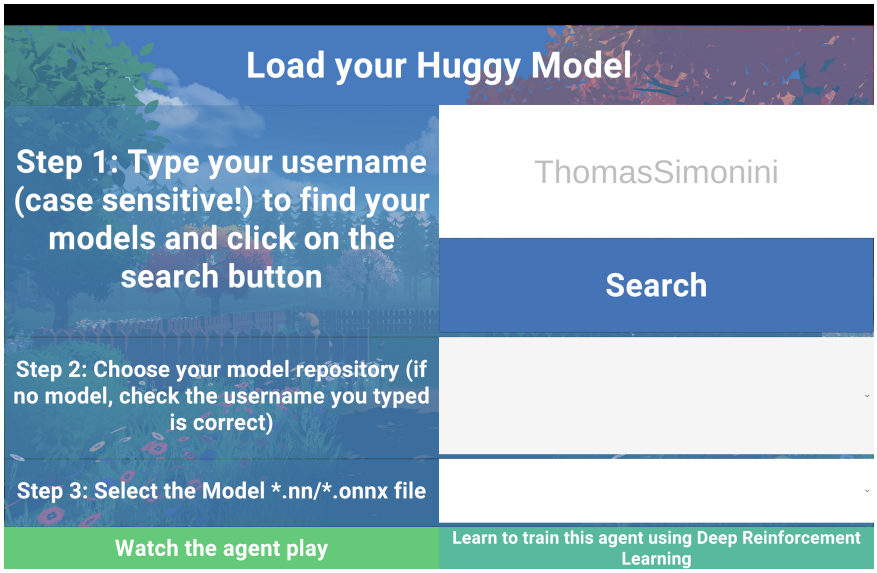


Figure 20: The Huggy browser demo: selecting a model from your account [original figure]

1. In step 1, choose your model repository which is the model id (in my case ThomasSimonini/ppo-Huggy).
2. In step 2, **choose which model you want to replay**:

- I have multiple ones, since we saved a model every 500000 timesteps.
- But if I want the most recent one I choose Huggy.onnx

It's good to **try with different model checkpoints to see the improvement of the agent.**

5 Conclusion

Congrats on finishing this bonus unit!

You can now sit and enjoy playing with your Huggy . And don't **forget to spread the love by sharing Huggy with your friends** . And if you share about it on social media, **please tag us @huggingface and me @simoninithomas**



Figure 21: Huggy the Dog in the game environment [original figure]

Finally, we would love **to hear what you think of the course and how we can improve it**. If you have some feedback then please fill out this form

Keep Learning, stay awesome